

P3823R0

Wording for US NB comment 10

Published Proposal, 2025-09-01

Author:

[Arthur O'Dwyer](#)

Audience:

WG21

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

US NB comment 10 objects to the new "identifiers with special meaning"

`trivially_relocatable_if_eligible` and `replaceable_if_eligible`. Compiler vendors will have to add their own keywords or attributes to deal with older language versions anyway (e.g. to get the optimizations for `unique_ptr` in C++20 mode); the new identifiers cause grammatical ambiguities; the new identifiers are laughably ugly; the new identifiers are not even necessary unless you are writing your own resource-management type in a C++26-only codebase.

Table of Contents

- 1 US NB comment 10**
- 2 Important note**
- 3 Proposed wording**
 - 3.1 [class.pre]
 - 3.2 [class.prop]
 - 3.3 [diff.cpp23.dcl.dcl]
 - 3.4 [diff.cpp23.library]

3.5 [lex.name]

References

Informative References

§ 1. US NB comment 10

The new identifiers with special meaning `trivially_relocatable_if_eligible` and `replaceable_if_eligible` are embarrassingly verbose. They are too long for practical use.

They will not see use in industry because their semantics do not match the semantics requested by over a dozen industry veterans representing seven major projects (Amadeus, Blender, Boost, Parlay, Folly, Qt, HPX) across [P3233], [P3236], [P1144R13], [P3780], etc.

Compiler vendors will have to add a `__keyword` or `[[attribute]]` anyway, to permit their library vendor to optimize types like `unique_ptr` in pre-C++26 modes. Library vendors cannot use the new identifiers with special meaning outside of C++26 mode.

Even in C++26 mode, a compiler-specific "opt-in" (P1144-style) attribute or keyword may be needed in order to support `constexpr optional` and `inplace_vector`, for which the P2786-style warrant is insufficient.

The new identifiers as such are useful only for user-defined resource-management classes, in C++26-only codebases. (Rule-of-Zero classes do not need them. C++23 codebases must use a vendor-specific mechanism such as an attribute.) This is a very small fraction of a small fraction: permitting this syntax gains very little in performance, but loses much in confusion to the industry and embarrassment to the Committee.

For all these reasons these identifiers should be removed from C++26, and their design reconsidered for C++29 after there has been any implementation and usage experience. We can safely remove these identifiers with special meaning, without at all harming the library feature of "relocation" itself.

§ 2. Important note

We needn't touch the library clauses; we needn't touch the traits; literally nothing in the Standard depends on these embarrassing and inutile keywords. (In fact, as US NB comment 9 points out, nothing in the Standard depends on the notion of "replaceability" either.)

§ 3. Proposed wording

The wording in this section is relative to the Committee Draft of C++26, [\[N5013\]](#).

§ 3.1. [\[class.pre\]](#)

Modify [\[class.pre\]](#) as follows:

class-property-specifier:

```
final
trivially_relocatable_if_eligible
replaceable_if_eligible
```

[...]

5. Each *class-property-specifier* shall appear at most once within a single *class-property-specifier-seq*. Whenever a *class-key* is followed by a *class-head-name*, the identifier `final`, ~~`trivially_relocatable_if_eligible`~~, or ~~`replaceable_if_eligible`~~, and a colon or left brace, the identifier is interpreted as a *class-property-specifier*.

[Example:

```
struct A;
struct A final {};           // OK, definition of struct A,
                             // not value-initialization of variable final

struct X {
    struct C { constexpr operator int() { return 5; } };
    struct B trivially_relocatable_if_eligible final : C{};
                             // OK, definition of nested class B,
                             // not declaration of a bit-field member
                             // trivially_relocatable_if_eligible final
};
```

—end example]

§ 3.2. [\[class.prop\]](#)

Modify [\[class.prop\]](#) as follows:

1. A *trivially copyable class* is a class:

- that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator ([special], [class.copy.ctor], [class.copy.assign]),
- where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and
- that has a trivial, non-deleted destructor ([class.dtor]).

2. A class C is *default-movable* if

- overload resolution for direct-initializing an object of type C from an xvalue of type C selects a constructor that is a direct member of C and is neither user-provided nor deleted,
- overload resolution for assigning to an lvalue of type C from an xvalue of type C selects an assignment operator function that is a direct member of C and is neither user-provided nor deleted, and
- C has a destructor that is neither user-provided nor deleted.

3. A class is *eligible for trivial relocation* unless it

- has any virtual base classes,
- has a base class that is not a trivially relocatable class,
- has a non-static data member of an object type that is not of a trivially relocatable type, or
- has a deleted destructor,

except that it is implementation-defined whether an otherwise-eligible union having one or more subobjects of polymorphic class type is eligible for trivial relocation.

4. A class C is a *trivially relocatable class* if it is eligible for trivial relocation and

- ~~has the trivially_relocatable_if_eligible_class_property_specifier,~~
- is a union with no user-declared special member functions, or
- is default-movable.

5. [Note: A class with const-qualified or reference non-static data members can be trivially relocatable. —end note]

6. A class C is *eligible for replacement* unless

- it has a base class that is not a replaceable class,
- it has a non-static data member that is not of a replaceable type,
- overload resolution fails or selects a deleted constructor when direct-initializing an object of type C from an xvalue of type C ([dcl.init.general]),
- overload resolution fails or selects a deleted assignment operator function when assigning to an lvalue of type C from an xvalue of type C ([expr.assign], [over.assign]), or
- it has a deleted destructor.

7. A class C is a *replaceable class* if it is eligible for replacement and

- ~~has the replaceable_if_eligible_class_property_specifier,~~
- is a union with no user-declared special member functions, or
- is default-movable.

8. [Note: Accessibility of the special member functions is not considered when establishing trivial relocatability or replaceability. —end note]

9. [Note: Not all trivially copyable classes are trivially relocatable or replaceable. —end note]

10. A class S is a *standard-layout class* if it: [...]

§ 3.3. [diff.cpp23.dcl.dcl]

Modify [diff.cpp23.dcl.dcl] as follows:

~~1. Affected subclause: [dcl.dcl.general]~~

~~Change: Introduction of trivially_relocatable_if_eligible and replaceable_if_eligible as identifiers with special meaning.~~

~~Rationale: Support declaration of trivially relocatable and replaceable types.~~

~~Effect on original feature: Valid C++ 2023 code can become ill-formed.~~

~~[Example:~~

```
struct C {};
struct C replaceable_if_eligible {}; // was well formed (new variable replaceable_if
// now ill formed (redefines C)
```

~~—end example]~~

§ 3.4. [diff.cpp23.library]

NOTE: This clause should point to [cpp.replace.general], not [res.on.macro.definitions]; that is the subject of US NB comment 57.

Modify [diff.cpp23.library] as follows:

2. Affected subclause: `[res.on.macro.definitions]`

Change: Additional restrictions on macro names.

Rationale: Avoid hard to diagnose or non-portable constructs.

Effect on original feature: Names of special identifiers may not be used as macro names. Valid C++ 2023 code that defines `replaceable_if_eligible` or `trivially_relocatable_if_eligible` as macros is invalid in this revision of C++.

§ 3.5. `[lex.name]`

Modify `[lex.name]` as follows:

2. The identifiers in Table 4 have a special meaning when appearing in a certain context. When referred to in the grammar, these identifiers are used explicitly rather than using the *identifier* grammar production. Unless otherwise specified, any ambiguity as to whether a given *identifier* has a special meaning is resolved to interpret the token as a regular *identifier*.

Table 4 — Identifiers with special meaning

<code>final</code>	<code>import</code>	<code>post</code>	<code>replaceable_if_eligible</code>
<code>override</code>	<code>module</code>	<code>pre</code>	<code>trivially_relocatable_if_eligible</code>

§ References

§ Informative References

[N5013]

Committee Draft — Programming Languages — C++. August 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5013.pdf>

[P1144R13]

Arthur O'Dwyer; et al. *std::is_trivially_relocatable*. May 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1144r10.html>

[P3233]

Giuseppe D'Angelo. *Issues with P2786 ('Trivial Relocatability For C++26')*. April 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3233r0.html>

[P3236]

Alan de Freitas; Daniel Liam Anderson; Giuseppe D'Angelo; Hans Goudey; Jacques Lucke; Krystian Stasiowski; Stéphane Janel; Thiago Maciera. *Please reject P2786 and adopt P1144*.

May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3236r1.html>

[P3780]

Giuseppe D'Angelo. *Detecting bitwise trivially relocatable types*. June 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3780r0.html>